hashlock.

Security Audit

Art Residences Property Fund Pty Ltd

January 2026 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	22
Conclusion	23
Our Methodology	24
Disclaimers	26
About Hashlock	27

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

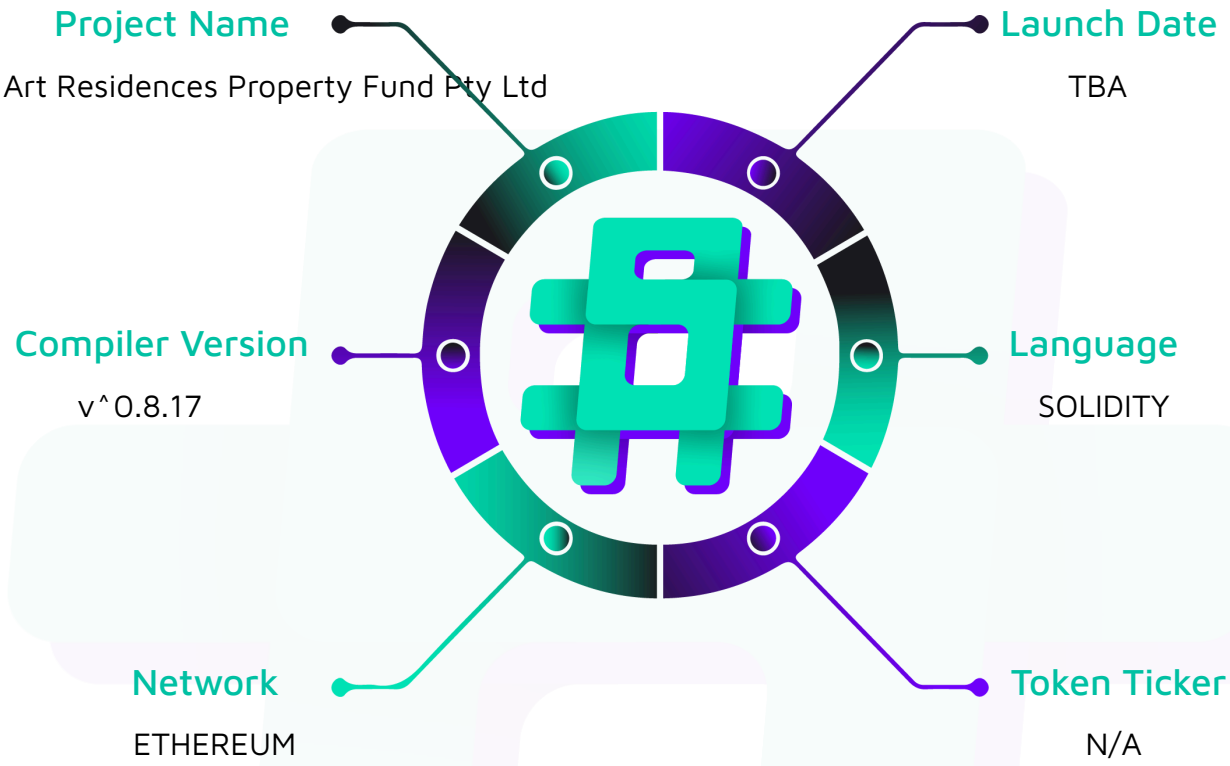
The Art Residences Property Fund Pty Ltd team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

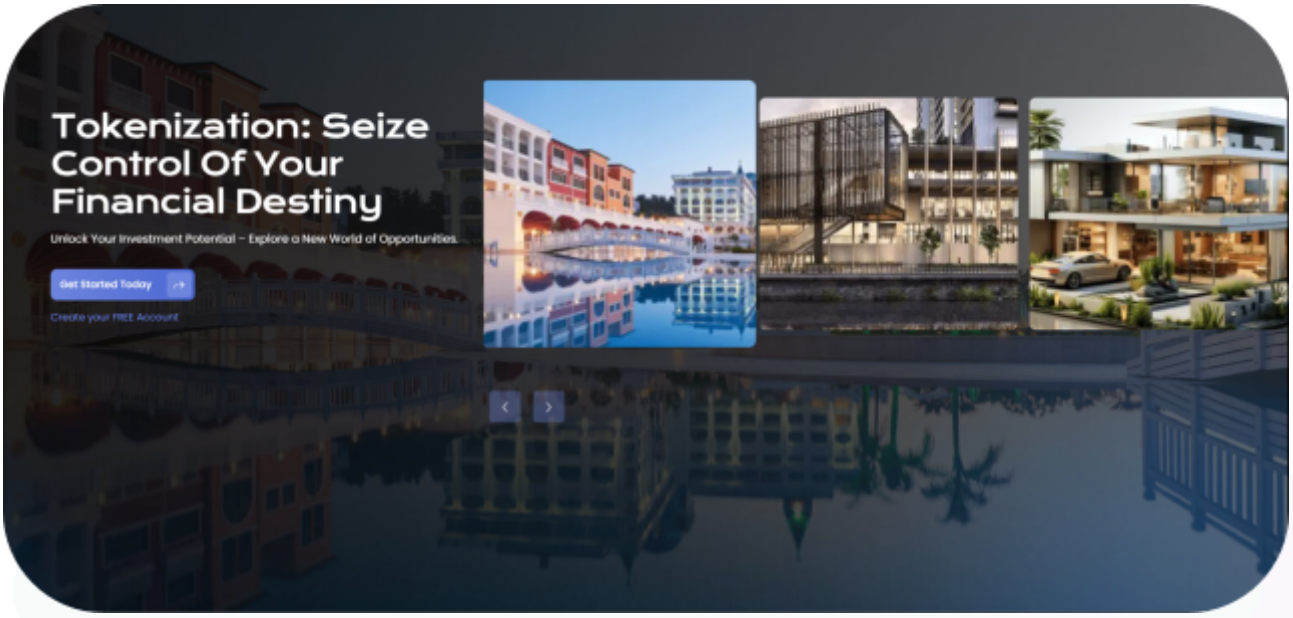
BT Asset Hub is an innovative digital platform developed by Black Tie Holdings Pty Ltd, designed to revolutionize traditional finance through asset tokenization. It enables asset managers to tokenize various asset classes—including real estate, mixed asset funds, lending and development funds, and businesses—by deploying smart contracts that convert physical assets into digital tokens. These tokenized assets are then listed on an integrated marketplace, where retail, wholesale, and institutional investors can access diverse investment opportunities using multiple payment methods, such as B4RC2, AUDD, USDT, and fiat currency. With its focus on compliance, security, and automated KYC/KYB processes, BT Asset Hub aims to democratize global investments, offering a user-friendly interface that empowers investors to seize control of their financial futures through blockchain technology.

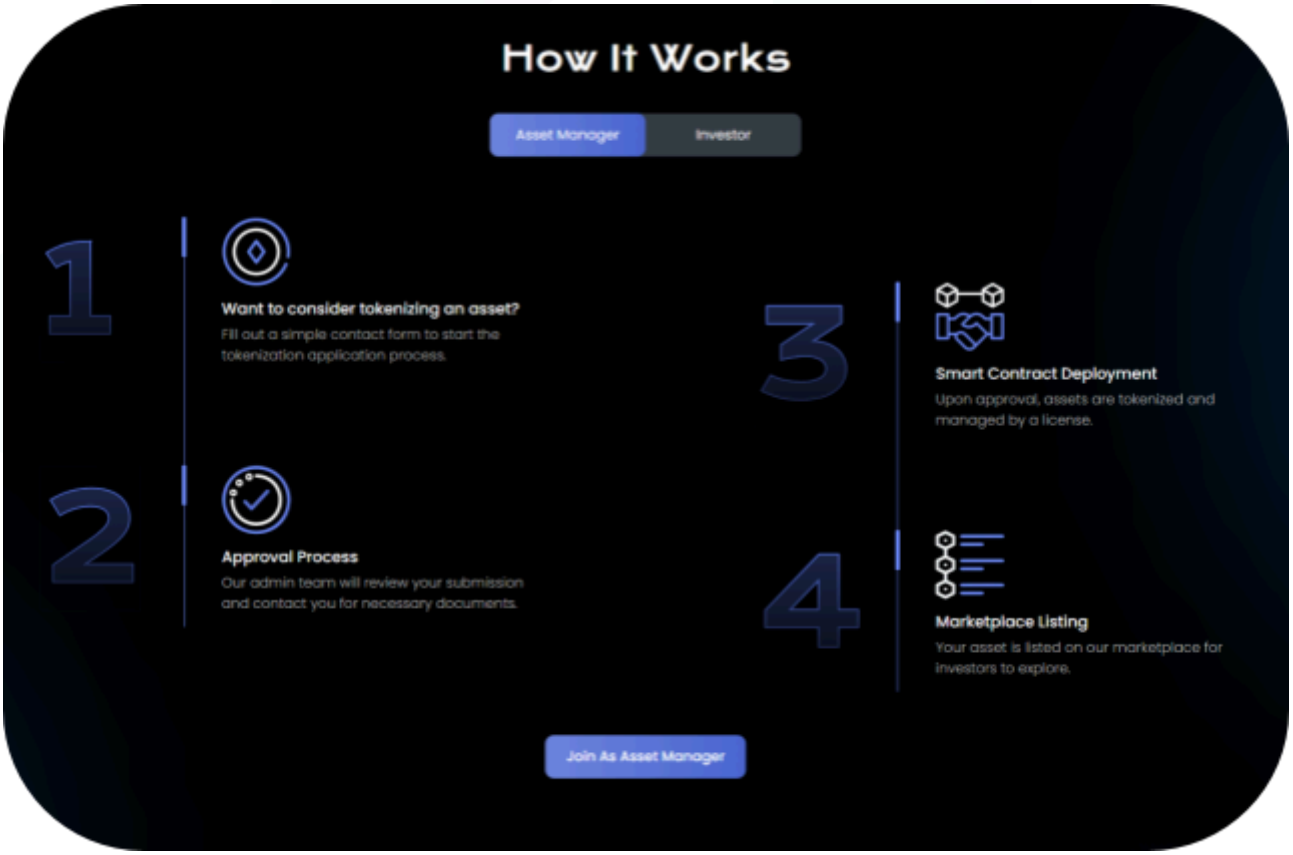
Project Name: Art Residences Property Fund Pty Ltd
Project Type: BaaS, Tokenization, Real Estate dApp, Lending Platform
Compiler Version: 0.8.17
Website: https://btassethub.digital/#homeSec
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the Art Residences Property Fund Pty Ltd project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Art Residences Property Fund Pty Ltd Smart Contracts
Platform	Ethereum / Solidity
Audit Date	January, 2026
Scope	https://etherscan.io/address/0x9Fc121B7e9a0ba2dF2e89bE6Bc56705d6baa864D#code

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been acknowledged.

Hashlock found:

3 Medium severity vulnerabilities

1 Low severity vulnerabilities

3 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour

Token.sol
- Allows the owner to:
  - Add and remove agents
  - Set token metadata
  - Set system dependencies
- Allows an agent to:
  - Pause and unpause transfers
  - Mint, burn
  - Force transfers, recovery
  - Freeze controls
  - Batch admin actions
- Allows users to:
  - Transfer tokens
  - Manage allowances
  - Batch transfers
	Contract achieves this functionality.

TokenProxy.sol
- Allows any caller to:
  - Call all token functions via proxy fallback
- Allows the implementation authority to:
  - Set a new ImplementationAuthority address
	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Art Residences Property Fund Pty Ltd project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Art Residences Property Fund Pty Ltd project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] Token#init - Implementation Contract Is Uninitialized

Description

The token implementation contract (`0x07c8…`) shows `owner() == 0x0` on-chain, implying it is not locked/initialized at the implementation address.

Vulnerability Details

`Token` uses OZ `initializer` but does not disable initializers in a constructor.

```
function init(
    address _identityRegistry,
    address _compliance,
    string memory _name,
    string memory _symbol,
    uint8 _decimals,
    // _onchainID can be zero address if not set, can be set later by owner
    address _onchainID
) external initializer {
    require(owner() == address(0), "already initialized");
    ...
    __Ownable_init();
    ...
}
```

Impact

A third party can likely call `init()` on the implementation itself (not the proxy) and become its owner. This does not directly grant control over proxy state, but it is considered unsafe by default and can break assumptions in tooling/ops.

Recommendation

In future versions, add a constructor in the implementation that calls `_disableInitializers()`.

Status

Acknowledged

[M-02] Token#transfer, transferFrom, mint, burn - External Compliance Callbacks Create Potential Reentrancy And DoS Surface

Description

Transfers, mints, burns invoke external functions on the compliance contract after state changes. This creates a real reentrancy surface whose exploitability depends on compliance behavior.

Vulnerability Details

Compliance callbacks are called during token flows:

```
_transfer(msg.sender, _to, _amount);

_tokenCompliance.transferred(msg.sender, _to, _amount);
```

Impact

A buggy or malicious compliance contract (or its modules) can cause reverts, recursion, unexpected gas usage, and potentially lock transfers.

Recommendation

Treat compliance (`0xf60F91…`) as an external dependency that can halt transfers instead of a passive checker. Independently verify its access control, upgradeability, and module set. If needed, add runtime guardrails like enforcing that compliance callbacks are non-reentrant.

Status

Acknowledged

[M-03] Token#setCompliance, setIdentityRegistry - Token Owner Can Brick The System Via Invalid Compliance Or IdentityRegistry Configuration

Description

`setCompliance` and `setIdentityRegistry` do not validate non-zero or contract code for the new addresses. Misconfiguration, accidental or malicious, can halt transfers and core functionality.

Vulnerability Details

No zero or contract code checks in setters:

```
function setIdentityRegistry(
    address _identityRegistry
) public override onlyOwner {
    _tokenIdentityRegistry = IIdentityRegistry(_identityRegistry);
    emit IdentityRegistryAdded(_identityRegistry);
}


function setCompliance(address _compliance) public override onlyOwner {
    if (address(_tokenCompliance) != address(0)) {
        _tokenCompliance.unbindToken(address(this));
    }
    _tokenCompliance = IModularCompliance(_compliance);
    _tokenCompliance.bindToken(address(this));
    emit ComplianceAdded(_compliance);
}
```

Impact

Setting either to `address(0)` or an EOA can cause `isVerified`, `canTransfer` calls to fail, effectively breaking transfers.

Recommendation

Add `require(_addr != address(0))`, `require(AddressUpgradeable.isContract(_addr))` or equivalent checks in future versions. Operationally, restrict these changes behind multisig and pre-validate target contracts before execution.

Status

Acknowledged

Low

[L-01] TokenProxy#fallback - Proxy Forwards `gas() - 10000`

Description

The proxy uses `delegatecall(sub(gas(), 10000), ...)`, which can cause avoidable failures as token, compliance logic grows in gas usage.

Recommendation

Forward full gas unless there is a measured reason not to, and test worst-case module configurations if the buffer is kept.

Status

Acknowledged

QA

[Q-01] Multiple Contracts - Hardhat Console Imported In Production Contracts

Description

`hardhat/console.sol` is imported in `TokenProxy.sol` and `Token.sol`, which is a development artifact and should not be shipped in production sources.

Recommendation

Remove `hardhat/console.sol` imports from deployable contract.

Status

Acknowledged

[Q-02] Token#batch* - Missing Array Length Validation In Batch Operations

Description

Batch functions index multiple arrays without checking equal lengths, causing out of bounds reverts and gas waste on mismatched inputs.

Recommendation

Add explicit length equality checks (e.g. `_toList.length == _amounts.length`) for each batch function.

Status

Acknowledged

[Q-03]

AbstractProxy#setImplementationAuthority,getImplementationAuthority - Selector Shadowing Constrains Future Upgrades

Description

The proxy defines `getImplementationAuthority()` and `setImplementationAuthority()`. Any future implementation function with the same selector would be unreachable through the proxy.

Recommendation

Maintain a reserved selector list and avoid adding colliding function signatures in future token implementations.

Status

Acknowledged

Centralisation

The Art Residences Property Fund Pty Ltd project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Art Residences Property Fund Pty Ltd project seems to have a sound and well-tested code base, now that our vulnerability findings have been acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au